

TELUGU VOICE CALLING AGENT

Build Your Own Inbound & Outbound AI Calling Agent — From Zero to Live Call

A complete, self-contained, step-by-step build guide using only free-tier and open-source tools. Follow it top to bottom with no other help needed.

WHAT YOU WILL HAVE BY THE END

- ✓ A phone number that answers calls automatically in Telugu
- ✓ A bot that understands speech, thinks, and replies naturally
- ✓ The same bot placing outbound calls to a list of numbers
- ✓ n8n automation logging every call and sending follow-ups

Guide version 1.0 • Free-tier demo build • Est. build time: 3–5 hours

Contents

Part 1	How This Guide Works & The Big Picture
Part 2	The Tools You Will Use (All Free-Tier)
Part 3	Before You Start — Accounts Checklist
Step 1	Create Your Telephony Account & Get a Number
Step 2	Get Telugu Speech Keys (Speech-to-Text & Text-to-Speech)
Step 3	Get a Free LLM Key (The Agent's Brain)
Step 4	Set Up Free Hosting for Your Bot
Step 5	Build the Voice Bot (Full Code Included)
Step 6	Connect Your Phone Number to the Bot (Inbound)
Step 7	Test Your First Inbound Call
Step 8	Build Outbound Calling
Step 9	Automate Everything with n8n (Free)
Step 10	Full End-to-End Test Checklist
Part 4	Free-Tier Limits Cheat Sheet
Part 5	Troubleshooting Guide
Part 6	Going From Demo to Production
Part 7	Glossary of Terms Used

Part 1 — How This Guide Works & The Big Picture

This guide is written so you can build a working Telugu voice calling agent using only this document — no other tutorials needed. Every step tells you exactly which website to open, what to click, what to copy, and what code to paste. Follow the steps in order without skipping.

Why a phone call needs more than one tool

A calling voice agent is not one piece of software — it is four small jobs happening in a fast loop, every time the caller speaks:

- 1 **Telephony** — actually connects the phone call and carries the audio (this guide uses Twilio).
- 2 **Speech-to-Text (STT)** — converts the caller's spoken Telugu into text (this guide uses Sarvam AI).
- 3 **The Brain (LLM)** — reads that text and decides what to say back (this guide uses Groq, free and fast).
- 4 **Text-to-Speech (TTS)** — converts the reply back into spoken Telugu audio (also Sarvam AI).

These four jobs are wired together by a small program called **Pipecat** (free, open-source), which runs on a free cloud server. **n8n** sits outside this real-time loop — it does not handle the live conversation. Instead it watches for "call finished" events and then does the business logic: saving the call to a spreadsheet, sending a WhatsApp/SMS follow-up, or starting the next outbound call in a list.

The flow, in plain words

Inbound call: Caller dials your number → Twilio answers and opens an audio stream → your bot (running on Render) hears the audio → Sarvam converts it to Telugu text → Groq's LLM decides a reply → Sarvam speaks the reply → Twilio plays it to the caller → repeat until the call ends → your bot notifies n8n.

Outbound call: n8n (or you, manually) tells Twilio "call this number" → Twilio dials it → once answered, it connects to the same bot → the rest of the loop is identical to inbound.

Important honest note: a no-code tool alone (including n8n by itself) cannot run the real-time voice loop — the back-and-forth is too fast for typical workflow-automation nodes. That is why this guide uses a small, free piece of code (about 90 lines) for the live conversation, and keeps n8n for everything else. This is exactly the split professional voice-AI agencies use.

Part 2 — The Tools You Will Use (All Free-Tier)

Job	Tool	Free tier (at signup)	Website
Telephony (calls)	Twilio	Free trial credit (~\$15) — enough for a full demo	twilio.com
Telugu STT + TTS	Sarvam AI	Free credits on signup for Indian-language STT/TTS	dashboard.sarvam.ai
LLM (the brain)	Groq Cloud	Generous free API tier, very fast responses	console.groq.com
Hosting your bot	Render.com	Free web service tier (sleeps when idle — fine for demo)	render.com
Automation	n8n	Free self-host, or free trial on n8n Cloud	n8n.io
Bot framework	Pipecat (open source)	100% free, no signup	github.com/pipecat-ai/pipecat

Free-tier limits change often. Before you begin, open each site above and check the current free-tier amount on their pricing page — this guide tells you where to look.

Part 3 — Before You Start: Accounts Checklist

Create these accounts first, in this order. Use the same email for all of them so it's easy to keep track.

- ■ Twilio account — twilio.com/try-twilio (needs a real phone number to verify)
- ■ Sarvam AI account — dashboard.sarvam.ai (sign up, verify email)
- ■ Groq Cloud account — console.groq.com (sign up with Google/GitHub is fastest)
- ■ Render.com account — render.com (sign up with GitHub — you'll need a GitHub account too)
- ■ GitHub account — github.com (free, needed to deploy your bot code to Render)
- ■ n8n — either install free on your own computer, or sign up for the n8n Cloud free trial

You do not need a credit card for Sarvam, Groq, Render, GitHub, or self-hosted n8n. Twilio may ask for card verification for full account features, but the trial credit itself works without spending real money.

STEP 1

Create Your Telephony Account & Get a Number

Goal: end this step owning a phone number that can receive and place calls.

- 1 Go to **twilio.com/try-twilio** and sign up with your email.
- 2 Verify your personal mobile number when asked (Twilio texts you a code).
- 3 You'll land on the Twilio Console. On the left menu, click **Phone Numbers** → **Manage** → **Buy a Number**.
- 4 Choose a country. Trial accounts can usually get one free number in your selected country (or a US number, which works fine for a demo). Filter by "Voice" capability.
- 5 Click **Buy** (it uses your trial credit, not real money).
- 6 Copy and save three things somewhere safe (a notes file): your **Account SID**, your **Auth Token** (both shown on the Console home page under "Account Info"), and your new **Twilio phone number**.
- 7 While your account is in trial mode, Twilio only allows calls *to* numbers you've verified. Go to **Phone Numbers** → **Verified Caller IDs** and add your own mobile number so you can call yourself to test.

For a real India production rollout later, replace Twilio with **Exotel** or **Ozonetel** — they handle Indian telecom compliance (DLT registration) that Twilio does not cover for Indian numbers. For this free demo, Twilio is simpler to set up in minutes.

STEP 2

Get Telugu Speech Keys

Goal: end this step with one API key that can turn Telugu speech into text, and text into Telugu speech.

- 1 Go to **dashboard.sarvam.ai** and create an account.
- 2 Once inside the dashboard, find **API Keys** in the left menu and click **Create API Key**.
- 3 Copy the key and save it in your notes file as `SARVAM_API_KEY`.
- 4 Check the dashboard's "Usage / Credits" page and note how many free credits you were given — this tells you roughly how many test minutes you can run.
- 5 Sarvam's Telugu language code is **te-IN** — you'll use this exact code in your bot's settings in Step 5.

Alternative if Sarvam's free credits run out: Soniox and Navana.ai's Bodhi TTS both also support Telugu and have their own signup flows with starter credits — the code in Step 5 can be swapped to any of these with minor changes since Pipecat supports multiple providers.

STEP 3

Get a Free LLM Key (The Agent's Brain)

Goal: a key that lets your bot ask an AI model what to say next.

- 1 Go to **console.groq.com** and sign up (Google/GitHub login is fastest).
- 2 Click **API Keys** in the left menu, then **Create API Key**.
- 3 Copy it and save it as GROQ_API_KEY.
- 4 Groq's free tier gives fast, free access to open models like Llama 3.1 — plenty for a demo agent's conversation logic.

Why Groq and not ChatGPT directly? Groq's free tier is generous and its responses are extremely fast, which matters a lot on a live phone call — a slow LLM makes the caller sit through awkward silence. You can swap in OpenAI's API later for production if you prefer.

STEP 4

Set Up Free Hosting for Your Bot

Goal: a live web address (URL) where your bot code runs 24/7, for free.

- 1 Create a free GitHub account at **github.com** if you don't have one.
- 2 Create a new empty repository, e.g. named telugu-voice-agent.
- 3 You will push your bot code (Step 5) into this repository.
- 4 Go to **render.com**, sign up, and click **New** → **Web Service**.
- 5 Connect your GitHub account and select the telugu-voice-agent repository.
- 6 Set the **Runtime** to Python 3, the **Start Command** to `python bot.py`, and choose the **Free** instance type.
- 7 Under **Environment**, add these four environment variables (this keeps your keys out of your code):
SARVAM_API_KEY, GROQ_API_KEY, TWILIO_ACCOUNT_SID, TWILIO_AUTH_TOKEN.
- 8 Don't click deploy yet — you'll do that after adding the code in Step 5.

Free Render web services "sleep" after 15 minutes of no traffic and take ~30–50 seconds to wake up on the next call. That's fine for a demo; for production, upgrade to a paid always-on instance so the first ring doesn't get missed.

STEP 5

Build the Voice Bot (Full Code Included)

Goal: the actual program that listens, thinks, and speaks in Telugu.

You do not need to be an expert coder — copy the files below exactly. This uses **Pipecat**, a free open-source framework built exactly for this kind of real-time voice pipeline.

5.1 — Create requirements.txt

```
pipecat-ai[websocket,sarvam,groq,twilio]
python-dotenv
fastapi
uvicorn
```

5.2 — Create bot.py

This is the core file. It sets up the four jobs from Part 1 and wires them into one pipeline that Twilio can talk to over a WebSocket.

```
import os
from dotenv import load_dotenv
from fastapi import FastAPI, WebSocket
from pipecat.pipeline.pipeline import Pipeline
from pipecat.pipeline.task import PipelineTask
from pipecat.pipeline.runner import PipelineRunner
from pipecat.transports.network.websocket_server import (
    WebSocketServerTransport, WebSocketServerParams)
from pipecat.services.sarvam import SarvamSTTService, SarvamTTSService
from pipecat.services.groq import GroqLLMService
from pipecat.processors.aggregators.llm_response import (
    LLMUserResponseAggregator, LLMAssistantResponseAggregator)

load_dotenv()
app = FastAPI()

SYSTEM_PROMPT = (
    "Nuvvu oka sahayaka voice agent vi. Telugu lo matladu, "
    "chinnaga mariyu [REDACTED] [REDACTED] [REDACTED]. Caller pettina prashnalaku "
    "sahayam cheyyi mariyu andamgaa maatladu."
)

async def run_bot(websocket: WebSocket):
    transport = WebSocketServerTransport(
        params=WebSocketServerParams(audio_in_enabled=True, audio_out_enabled=True))

    stt = SarvamSTTService(api_key=os.getenv("SARVAM_API_KEY"), language="te-IN")
    tts = SarvamTTSService(api_key=os.getenv("SARVAM_API_KEY"), language="te-IN",
                           voice="meera")
    llm = GroqLLMService(api_key=os.getenv("GROQ_API_KEY"), model="llama-3.1-8b-instant")

    messages = [{"role": "system", "content": SYSTEM_PROMPT}]
    user_agg = LLMUserResponseAggregator(messages)
    asst_agg = LLMAssistantResponseAggregator(messages)

    pipeline = Pipeline([
        transport.input(),
        stt,
        user_agg,
        llm,
        tts,
```

```

        transport.output(),
        asst_agg,
    ])

    task = PipelineTask(pipeline)
    runner = PipelineRunner()
    await runner.run(task)

@app.websocket("/ws")
async def websocket_endpoint(websocket: WebSocket):
    await websocket.accept()
    await run_bot(websocket)

if __name__ == "__main__":
    import uvicorn
    uvicorn.run(app, host="0.0.0.0", port=int(os.getenv("PORT", 8000)))

```

What this code does, line by line in plain English: it opens a live audio connection (WebSocket), sends incoming audio to Sarvam to become Telugu text, adds that text to a running conversation, sends the conversation to Groq's LLM for a reply, converts the reply back to Telugu speech with Sarvam, and streams it back down the same connection. The `SYSTEM_PROMPT` is written in Telugu/Romanized-Telugu so the model replies in Telugu — edit this line to change your agent's personality or purpose.

5.3 — Push it to GitHub, then deploy

- 1 In your project folder, run: `git init`, then `git add .` && `git commit -m "first version"`.
- 2 Follow GitHub's on-screen instructions to link this to your empty repository and push it.
- 3 Go back to your Render web service and click **Manual Deploy** → **Deploy latest commit**.
- 4 Wait for the build to finish (a few minutes). Render will show you a live URL like `https://telugu-voice-agent.onrender.com` — save this.

STEP 6

Connect Your Phone Number to the Bot (Inbound)

Goal: make an incoming call to your Twilio number reach your bot.

- 1 In the Twilio Console, go to **Phone Numbers** → **Manage** → **Active Numbers** and click your number.
- 2 Scroll to **Voice Configuration**. Under "A call comes in", choose **Webhook**.
- 3 Twilio needs a small piece of TwiML (its call-instruction format) that tells it to open a media stream to your bot. The easiest way for a demo: use a **TwiML Bin** — in the Twilio Console, go to **Developer Tools** → **TwiML Bins** → **Create new TwiML Bin** and paste:

Replace the URL with your own Render URL, and note it starts with wss:// (a "secure WebSocket"), not https://.

- 1 Save the TwiML Bin, copy its URL, then go back to your phone number's Voice Configuration and select this TwiML Bin as the webhook for "A call comes in".
- 2 Save the number configuration.

STEP 7

Test Your First Inbound Call

Goal: hear your bot answer and reply in Telugu.

- 1 From your own verified mobile number, call your Twilio number.
- 2 You should hear a short connection delay (longer the first time if Render was asleep), then your bot should greet you.
- 3 Speak a simple Telugu sentence and wait — the bot should transcribe it, think, and reply.
- 4 If nothing happens: open your Render service's **Logs** tab in another window while you call — errors will appear there in real time (see Part 5, Troubleshooting, if you get stuck).

Expect the first couple of test calls to be rough — a wrong voice setting, a typo'd key, or a sleeping server are the most common first-run issues, not a fundamental problem with your setup.

STEP 8

Build Outbound Calling

Goal: make your bot call out to a number instead of waiting for a call in.

Outbound uses the exact same bot — the only difference is who starts the call. Instead of a caller dialing in, you tell Twilio's API to dial out, and point that outbound call at the same TwiML Bin from Step 6.

Create `make_call.py`:

```
import os
from twilio.rest import Client

client = Client(os.getenv("TWILIO_ACCOUNT_SID"), os.getenv("TWILIO_AUTH_TOKEN"))

call = client.calls.create(
    to="+91XXXXXXXXXX",          # the number to call
    from_="+1XXXXXXXXXX",        # your Twilio number
    url="https://handler.twilio.com/twiml/EHxxxxxxxxxxxxx" # your TwiML Bin URL
)
print("Calling... Call SID:", call.sid)
```

- 1 Run `pip install twilio` on your own computer (or add "twilio" to your requirements.txt if running this on Render too).
- 2 Replace the to number with a number you've verified in trial mode, and the url with your TwiML Bin's own URL (found on the TwiML Bin's page in Twilio).
- 3 Run `python make_call.py` — your phone should ring within a few seconds, connecting you to the same Telugu bot.
- 4 For calling a list of numbers, wrap the `client.calls.create(...)` line in a loop over a list, adding a short delay between calls to avoid rate limits.

STEP 9

Automate Everything with n8n (Free)

Goal: automatically log every call and trigger follow-ups, with zero manual work.

This is where n8n does what it's actually good at. You'll add one line to bot.py that "pings" an n8n webhook when a call ends, with a summary of what happened.

9.1 — Install n8n for free

- Easiest: sign up for the n8n Cloud free trial at **n8n.io** — no install needed.
- Fully free forever option: install n8n on your own computer with `npx n8n` (requires Node.js) or via Docker — both are free, self-hosted, unlimited.

9.2 — Build the workflow

- 1 In n8n, create a new workflow and add a **Webhook** node as the trigger. Set it to accept POST requests. Copy the webhook's **Test URL**.
- 2 Add a **Google Sheets** node after it, set to "Append Row", so every call gets logged with caller number, timestamp, and summary.
- 3 Optionally add a **Twilio** or **WhatsApp Business Cloud** node after that to send a follow-up SMS/WhatsApp message automatically.
- 4 Click **Save** and **Activate** the workflow so the webhook stays listening.

9.3 — Call the webhook from your bot

Add this near the end of your `run_bot` function in `bot.py`, once the call/pipeline finishes:

```
import requests

def notify_n8n(caller_number, summary):
    requests.post(
        "https://YOUR-N8N-WEBHOOK-URL",
        json={"caller": caller_number, "summary": summary}
    )
```

Call `notify_n8n(...)` right after `await runner.run(task)` finishes, passing in whatever caller details and conversation summary you have available at that point.

This one webhook is the entire bridge between your real-time voice bot and your no-code automation — everything downstream (spreadsheets, CRM, WhatsApp, follow-up scheduling) now lives safely inside n8n where you can change it without touching the bot's code again.

STEP 10

Full End-to-End Test Checklist

Run through this list before you call it done.

- ■ Inbound: calling your Twilio number connects to the bot within ~1 minute (allow for cold start).
- ■ The bot's Telugu greeting plays clearly.
- ■ Speaking Telugu is transcribed correctly (check Render logs to confirm the STT text).
- ■ The LLM's reply makes sense and stays in Telugu.
- ■ The reply is spoken back in a natural-sounding Telugu voice.
- ■ Outbound: running `make_call.py` successfully calls your own verified number.
- ■ After the call ends, a new row appears in your Google Sheet within a few seconds.
- ■ (If configured) a follow-up SMS/WhatsApp message arrives after the call.

Part 4 — Free-Tier Limits Cheat Sheet

These are typical free-tier shapes at the time of writing — always double-check the live pricing page before you rely on a number, since free tiers change often.

Tool	What the free tier typically gives you	What happens after it runs out
Twilio	One-time trial credit (a few dollars) covering roughly 60–100 test minutes; only calls to verified numbers	Add a card and top up, or move India numbers to Exotel
Sarvam AI	Free starter credits on signup for STT/TTS	Pay-as-you-go per character/minute
Groq Cloud	Free API tier with rate limits (requests per minute)	Move to a paid tier or another LLM provider
Render.com	Free web service, sleeps after 15 min idle, limited monthly hours	Upgrade to a paid "always-on" instance
n8n	Self-hosted = free forever; Cloud = free trial only	Keep self-hosting for free, or subscribe to Cloud

Part 5 — Troubleshooting Guide

Call connects but there's silence

Check Render logs — usually a wrong API key or the Sarvam language code isn't set to te-IN.

Call drops after a few seconds

The WebSocket URL in your TwiML Bin is wrong, or it's using ws:// instead of wss://.

Bot replies in English, not Telugu

Double-check the SYSTEM_PROMPT language and that the TTS voice/language parameter is set to Telugu, not left at a default.

First call takes 30–50 seconds to connect

Normal on Render's free tier — the server was asleep and is waking up. Upgrade to a paid instance to remove this delay.

Twilio says the number isn't verified

In trial mode Twilio can only call numbers you've added under Verified Caller IDs — add the number first.

n8n webhook never receives anything

Make sure the workflow is **Activated** (not just saved), and that you're using the Production URL, not the Test URL, once you're done testing.

Groq returns a rate-limit error

Free tier has a requests-per-minute cap — add a short retry/delay, or reduce how often the bot calls the LLM mid-conversation.

Part 6 — Going From Demo to Production

Once your demo works, these are the specific upgrades agencies make before charging real customers:

- 1 **Swap Twilio for Exotel or Ozonetel** for Indian numbers — they handle DLT/TRAI compliance that Twilio does not for India-originated traffic.
- 2 **Move off Render's free tier** to an always-on paid instance so there's no cold-start delay on the first ring.
- 3 **Add call recording and consent messaging** — required by Indian telecom regulations for many use cases; play a short recorded-call disclosure at the start of the call.
- 4 **Add a human-handoff path** — if the bot can't help, transfer the call to a real agent's number using Twilio/Exotel's call-transfer feature.
- 5 **Load-test your Render/hosting instance** — a paid instance with more memory handles multiple simultaneous calls; the free tier will choke under real call volume.
- 6 **Move your LLM to a paid, higher-quality model** if Groq's free tier's occasional rate limits become a problem at volume.
- 7 **Add monitoring** — pipe call transcripts and errors into a dashboard (or an n8n workflow that alerts you) so failures don't go unnoticed.

Part 7 — Glossary of Terms Used in This Guide

STT (Speech-to-Text)	Software that converts spoken audio into written text.
TTS (Text-to-Speech)	Software that converts written text into spoken audio.
LLM (Large Language Model)	The AI model that reads text and generates a reply, e.g. Llama or GPT.
Webhook	A URL that another service calls automatically when something happens (e.g. "call ended").
WebSocket	A connection that keeps sending data both ways continuously — needed for live audio, unlike a normal one-time web request.
Twiml	Twilio's simple XML-based language for telling it what to do with a call.
API key	A private password-like code that lets your program use a service like Sarvam or Groq.
Cold start	The delay a free server takes to "wake up" after being idle.
DLT / TRAI	India's telecom regulations governing commercial calling and messaging.

End of guide. Build it in order, test after every step, and don't move on until each one works — that's the whole method.